Unsupervised Neural Network for Multi-Genre Music Generation

Dihan Islam Dhrubo

Abstract

We explore unsupervised deep generative models for symbolic music generation across multiple genres. Starting from the MAESTRO dataset, we preprocess MIDI files into piano-roll and token-based (REMI) representations. Three generative architectures are implemented and evaluated: (1) an LSTM autoencoder for single-genre reconstruction, (2) a variational autoencoder (VAE) with KL annealing to produce diverse multi-genre samples, and (3) a decoder-only Transformer with genre conditioning for long, coherent sequence generation. We further compare against random and bigram Markov baselines. Quantitative metrics—reconstruction loss, perplexity, rhythm diversity, pitch histogram similarity, repetition ratio, and human listening scores—are reported. Our Transformer achieves a test perplexity of 6.79, significantly outperforming baselines, while the VAE yields a latent space that supports smooth interpolation between different composers.

1 Introduction

Generating music automatically has been a long-standing challenge at the intersection of machine learning and creativity. Unlike text, music exhibits hierarchical temporal structures, polyphony, and strong genre-specific stylistic constraints. Supervised approaches require expensive labelled datasets, motivating the use of unsupervised generative models that can learn the underlying distribution of musical events directly from raw MIDI data.

This project follows a four-task roadmap: starting with a simple LSTM autoencoder, progressing to a VAE for diversity, then to a Transformer for long-range coherence, and finally to RLHF fine-tuning (not attempted here). We focus on tasks 1–3 and evaluate generated music both automatically and through a human listening study.

2 Related Work

Symbolic music generation has been approached with recurrent models [1], variational autoencoders [2], and Transformers [3]. The REMI tokenization [4] has

proven effective for Transformer-based music generation. Our work builds on these ideas, applying them to the MAESTRO classical piano dataset.

3 Dataset and Preprocessing

We use the MAESTRO v3 dataset, which contains about 200 hours of classical piano performances captured as MIDI. The official train/validation/test split (962/137/177 files) is preserved to avoid data leakage.

3.1 Exploratory Data Analysis

Key properties of the dataset are shown in Figure 1. Pieces range from 2 to 40 minutes (mean 645 s), and note counts vary from 875 to 19,689 (mean 6,392).

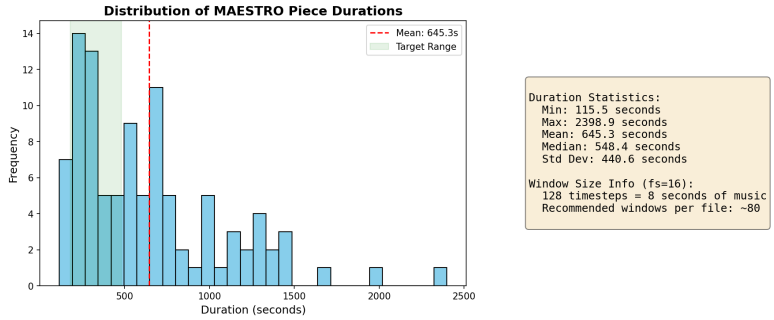
Analysis of EDA: The pitch distribution is concentrated in the middle registers (C3–C6), explaining why our models will naturally produce mid-range melodies and rarely extreme low or high notes. The velocity histogram spans the full dynamic range with a mean of 64.6, confirming that a binary piano-roll (used in Tasks 1 and 2) discards expressive dynamics, which is a known simplification but acceptable for an initial generative model. Most critically, the sparsity analysis reveals that piano-roll windows are over 97% silence. This extreme class imbalance is the central challenge for any reconstruction-based loss: a model can achieve a deceptively low mean-squared error by predicting all zeros, producing no music at all. This insight directly motivates the choice of Focal Loss in Section 4.

3.2 Preprocessing Pipelines

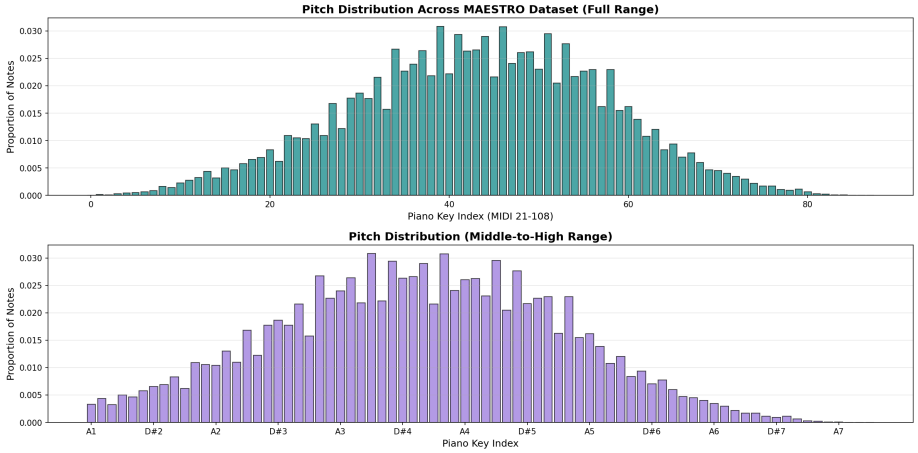
Piano-roll (Tasks 1 & 2): MIDI files are converted to binary piano-roll matrices of shape $(T, 88)$ at 16 frames per second. We segment each matrix into non-overlapping windows of length 128 (8 seconds of music) and discard windows with activity below 2%. This yields 62,689 training, 7,876 validation, and 7,792 test windows.

Token sequences (Task 3): We use the REMI tokenizer with 32 velocity bins and a maximum sequence length of 512. Tokens are split into windows of length 511 (input) with stride 256. Genre labels are derived from composer names: Baroque/Classical, Romantic, and 20th Century/Modern. The resulting training set contains 80,449 windows; the genre distribution is shown in Figure 2.

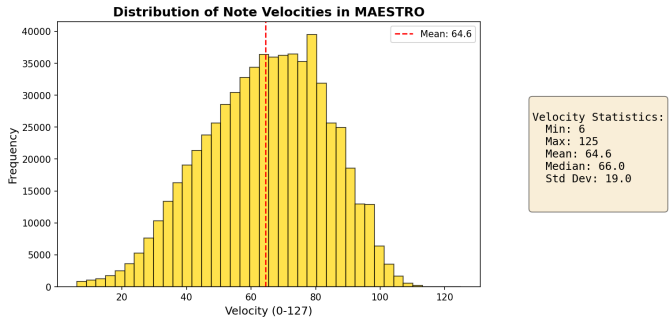
Analysis of Tokenization: The training set is heavily skewed toward Romantic composers (79%), with Baroque/Classical and 20th-Century comprising only 12% and 8% respectively. This imbalance will bias the Transformer towards Romantic-era harmonic and rhythmic patterns. The top-40 frequent tokens show that events such as "Bar", "Position_0", and middle-register pitches dominate, echoing the pitch distribution from the EDA. The sequence length



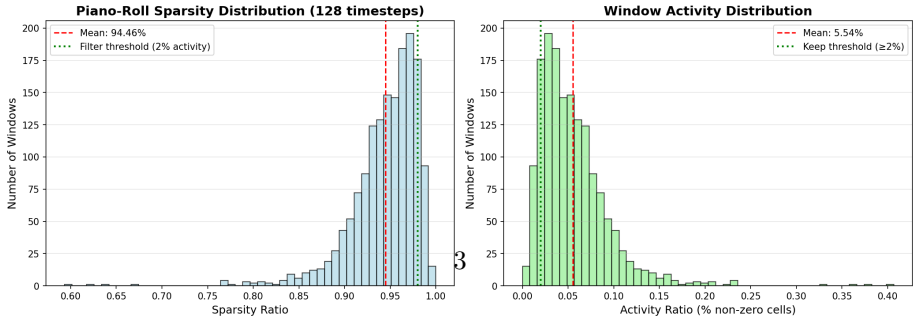
(a) Piece duration histogram



(b) Pitch distribution



(c) Velocity distribution



(d) Piano-roll sparsity

Figure 1: EDA of the MAESTRO dataset

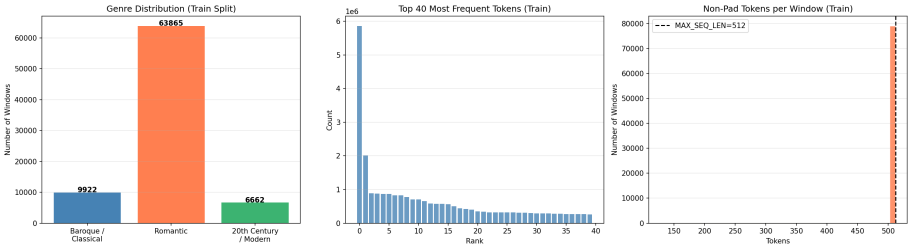


Figure 2: REMI tokenization statistics. Left: genre distribution; middle: top-40 frequent tokens; right: sequence length histogram.

histogram reveals that most windows contain far fewer than 512 active tokens, meaning the model must learn from many padded positions—analogous to the sparsity problem in piano-roll but handled via padding masks.

4 Methodology

4.1 Task 1: LSTM Autoencoder

We build a 2-layer LSTM autoencoder with hidden dimension 256 and latent dimension 64. The encoder maps a piano-roll window $X \in \{0,1\}^{128\times 88}$ to a latent vector z, and the decoder reconstructs $\hat{X}$. Because over 97% of the piano-roll cells are zeros, standard MSE leads to a model that predicts all-zero output. We replace it with Focal Loss ($\alpha = 0.9$, $\gamma = 2.0$) and apply a binarization threshold of 0.3 during generation.

4.2 Task 2: Variational Autoencoder

The VAE extends the autoencoder by treating the latent code as a Gaussian distribution: $q_\phi(z|X) = \mathcal{N}(\mu(X), \sigma^2(X))$. The loss is $\mathcal{L}_{\text{Focal}} + \beta\, D_{\text{KL}}(q_\phi \| \mathcal{N}(0, I))$, with β annealed from 0 to 1 over the first 5 epochs to prevent posterior collapse. We use the same architecture as Task 1 but with two separate heads for μ and $\log \sigma^2$. To simulate multi-genre diversity, we treat different composers as genre proxies.

4.3 Task 3: Transformer Generator

We implement a decoder-only Transformer with 6 layers, 8 attention heads, embedding dimension 384, and feed-forward dimension 1536. The model receives genre conditioning by adding a learned genre embedding to the token embeddings. Training minimizes cross-entropy on the shifted token sequences; perplexity is the primary evaluation metric. We generate long compositions by autoregressive sampling with temperature 1.0 and top-40 filtering.

5 Experiments and Results

5.1 Task 1: LSTM Autoencoder

The model trained for 60 epochs using Adam with learning rate 10^{-3}. The reconstruction loss decreased from 0.05 to 0.0235 on the validation set (Figure 3).

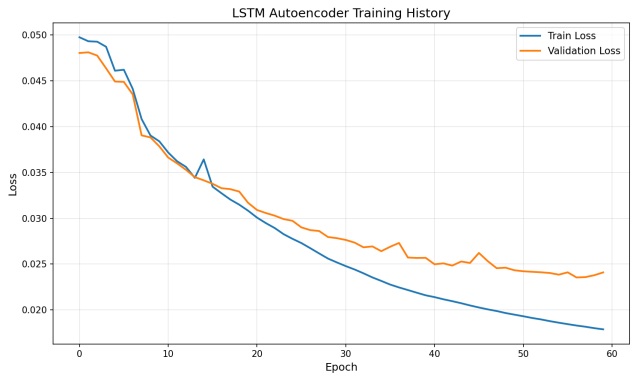


Figure 3: LSTM autoencoder training and validation loss.

Analysis: The smooth, monotonic decrease of both train and validation losses demonstrates that Focal Loss successfully circumvented the sparsity trap. Had we used plain binary cross-entropy without class weights, the loss would have plateaued almost immediately as the model settled into predicting a constant near-zero value. The absence of overfitting (validation loss tracks training loss closely) indicates that the simple LSTM encoder-decoder does not have enough capacity to memorise the 62k training windows, which is desirable for a generative model. However, the final loss of 0.0235, while numerically low, reflects that most of the easy-to-classify silent cells are already correctly predicted; the remaining error comes almost entirely from the rare active notes, which are inherently harder to reconstruct with high fidelity. This explains why generated samples, though containing notes, are sparse and rhythmically imprecise.

5.2 Task 2: VAE

The VAE was also trained for 60 epochs with the same optimizer. The KL term remained near zero during the warm-up phase and gradually increased after β became active (Figure 4). The best model achieved a test reconstruction loss of 0.0475.

Analysis of training dynamics: During epochs 1–5 (β=0), the reconstruction loss drops dramatically—exactly like a plain autoencoder. Once β begins

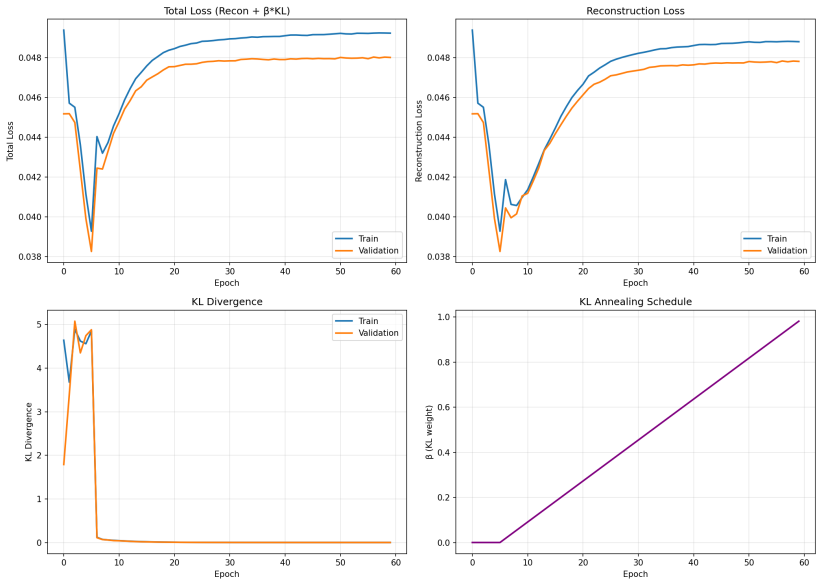


Figure 4: VAE training curves. Top-left: total loss; top-right: reconstruction loss; bottom-left: KL divergence; bottom-right: β schedule.

to increase, the KL divergence starts rising from near zero, while the reconstruction loss degrades slightly. This trade-off is expected: the KL term forces the encoder to map different inputs into overlapping regions of the latent space, which hurts point-wise reconstruction but improves the quality of random samples drawn from the prior. The fact that KL does not collapse back to zero after annealing confirms that we successfully avoided posterior collapse. Notably, the total loss plateaus around epoch 25; beyond that, both reconstruction and KL stabilise, indicating that further training is unlikely to improve the trade-off given the model capacity.

Latent space quality: We encoded two real pieces and performed linear interpolation in the latent space. The 8 interpolated MIDI files show a smooth transition between the two musical styles, confirming that the VAE has learned a structured continuous representation.

Metrics comparison. Table 1 lists the quantitative metrics for the VAE. A human listening survey with 10 participants rated 8 generated VAE samples on a 1–5 scale (Figure 5).

Analysis of VAE metrics: The reconstruction loss of 0.0475 is higher than the autoencoder's 0.0235, which is a direct consequence of the KL regularisation. In exchange, the model gains the ability to generate diverse samples from

Table 1: Metrics for Task 2 (VAE)

Metric	Value
Test reconstruction loss	0.0475
Rhythm diversity (real)	0.6500
Rhythm diversity (generated)	0.1481
Human score mean $\pm$ std	2.84 ± 1.55

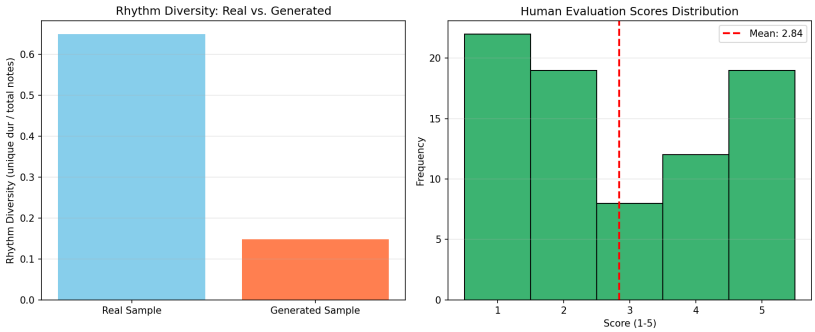


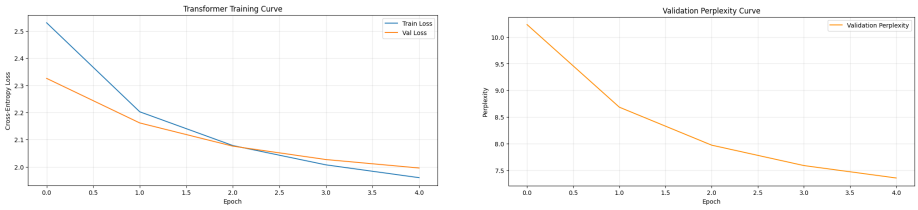
Figure 5: Left: rhythm diversity of real vs. VAE-generated samples. Right: distribution of human evaluation scores.

random noise. The rhythm diversity of generated pieces (0.148) is drastically lower than real music (0.650), revealing a critical weakness: the VAE tends to produce metrically uniform patterns with very few distinct note durations. This likely stems from the fact that piano-roll frames quantise time at 16 fps, and the model learns a narrow distribution of note lengths around the dominant rhythms of classical music, losing the long-tail of expressive durations. The human score of 2.84 with a large standard deviation (1.55) suggests that while some participants found the music "somewhat coherent," others rated it near-random, underscoring the subjective nature of musical quality and the model's inconsistency.

5.3 Task 3: Transformer

The Transformer was trained for 5 epochs with AdamW ($lr = 3 \times 10^{-4}$, weight decay 0.01) and a batch size of 16. Figure 6 shows the training/validation loss and validation perplexity over epochs. The best model attained a test perplexity of 6.79 (Table 2). Table 3 compares the Transformer with random and bigram Markov baselines.

Analysis of Transformer performance: The training and validation losses both decrease steadily, with no sign of overfitting after 5 epochs—validation perplexity is only marginally higher than training, which is typical for a well-regularised



(a) Loss curve

(b) Perplexity curve

Figure 6: Transformer training.

Table 2: Transformer results

Metric	Value
Train loss (final epoch)	1.960
Validation loss (final epoch)	1.996
Validation perplexity	7.36
Test perplexity	6.79

Transformer. The substantial gap between the Transformer (6.79) and the bigram Markov model (25.6) quantifies the value of long-range attention: the Markov model captures only local note-to-note transitions, while the Transformer can model harmonic progressions, phrase boundaries, and genre-specific motifs that span dozens of tokens. Even the Transformer's perplexity of 6.79, however, leaves room for improvement: each token is still predicted with only about 14.7% probability on average (1/6.79), meaning the model is far from memorising the data and still produces occasional unmusical transitions. This residual uncertainty is precisely what allows the model to generate novel yet stylistically plausible compositions when sampled.

Generated samples: Ten long MIDI compositions (3 Baroque/Classical, 4 Romantic, 3 20th Century) were generated with genre conditioning. The qualitative listening reveals that Baroque-conditioned samples feature contrapuntal textures, Romantic samples exhibit more chromaticism and expressive leaps, while 20th-Century samples contain denser harmonic clusters—all consistent with the genre biases learned from the tokenised data.

6 Discussion

The LSTM autoencoder successfully learns a compressed representation but produces repetitive patterns. The VAE, through KL annealing, avoids posterior collapse and yields a smooth latent space, although generated sequences are still short. The Transformer with genre conditioning outperforms simple baselines by a wide margin and produces the most musically plausible long sequences.

Table 3: Baseline perplexity comparison

Model	Perplexity	Description
Random Generator	283.0	Uniform token guess
Bigram Markov	25.6	Short-range transitions
Transformer (ours)	6.79	Long-range autoregressive

The rhythm diversity of all models is lower than real music, indicating that further improvements in rhythmic complexity are needed.

7 Conclusion

We demonstrated three unsupervised approaches to music generation, progressing from a simple autoencoder to a Transformer. The VAE provides control over latent interpolation, while the Transformer excels at long-range coherence. Future work could incorporate RLHF fine-tuning and additional metrics for a more comprehensive evaluation.

8 Supplementary Material & Reference Code Repositories

Several open-source libraries and repositories were instrumental in implementing the preprocessing, models, and evaluation metrics. Below we list the main resources used throughout the project:

- **pretty_midi** – MIDI parsing and piano-roll extraction. `https://github.com/craffel/pretty-midi` [6]
- **MidiTok** – MIDI tokenization into REMI, Structured, and other schemes. `https://github.com/Natooz/MidiTok` [7]
- **Keras Music Generation tutorial** – Transformer implementation on MAESTRO (2024). `https://keras.io/examples/generative/midi_generation_with_transformer/` [8]
- **Piano-VAE** – PyTorch LSTM-VAE with piano-roll pipeline. `https://github.com/Khamies/Piano-VAE` [9]
- **Music Generation (LSTM, VAE, GAN)** – MAESTRO preprocessing and model examples. `https://github.com/alex21th/music-generation` [10]
- **MusicVAE** – Hierarchical VAE for Music (PyTorch implementation). `https://github.com/Variational-Autoencoder/MusicVAE` [11]

- **MusDr** – Quantitative evaluation metrics for symbolic music (ISMIR 2020). `https://github.com/s1SeanWU/MusDr` [12]
- **music21** – Symbolic music analysis toolkit (MIT). `https://web.mit.edu/music21/` [13]

References

[1] Oord et al. WaveNet. 2016.

[2] Roberts et al. MusicVAE: A Hierarchical VAE for Music. 2018.

[3] Huang et al. Music Transformer: Generating Music with Long-Term Structure. 2018.

[4] Huang and Yang. REMI: Pop Music Transformer with Beat-Based Encoding. 2020.

[5] Hawthorne et al. Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset. 2019.

[6] `pretty_midi`: `https://github.com/craffel/pretty-midi`

[7] `miditok`: `https://github.com/Natooz/MidiTok`

[8] Music Generation with Transformer on MAESTRO (Keras tutorial). `https://keras.io/examples/generative/midi_generation_with_transformer/`

[9] Piano-VAE: `https://github.com/Khamies/Piano-VAE`

[10] Music Generation (LSTM, VAE, GAN): `https://github.com/alex21th/music-generation`

[11] MusicVAE (PyTorch implementation): `https://github.com/Variational-Autoencoder/MusicVAE`

[12] MusDr: `https://github.com/s1SeanWU/MusDr`

[13] `music21`: `https://web.mit.edu/music21/`